# Cool Roofing — Quick Reference Cheat Sheet

**SIH 2024 | All 12 Slides + Key Terms**

## Slide 1 — Title / Overview

- System: 9 inputs → 5 ML predictions in < 1 second
- Dataset: 2,500 building records
- Models: XGBoost (cost) + Random Forest (4 others)
- App: Streamlit — 5 pages
- Star metric: **98.9% cost prediction accuracy ($R^2 = 0.9843$)**

## Slide 2 — Problem

- No pre-install cost estimate → ₹1L to ₹18L guessing range
- No energy savings forecast before installation
- No temperature drop prediction
- 15 method × quality combos chosen by gut, not data
- We solve all 4 with one ML system

## Slide 3 — Proposed Solution

| Output | Model | Accuracy |
|---|---|---|
| Installation Cost | XGBoost | $R^2$ = 0.9843, MAE ₹69K |
| Temperature Drop | Random Forest | $R^2$ = 0.935, MAE 0.46°C |
| Units Saved/month | Random Forest | $R^2$ = 0.944, MAE 5.41 kWh |
| High Rating ≥4★ | RF Classifier | 65.5%, AUC 0.728 |
| Customer Referral | RF Classifier | 62.0%, F1 0.505 |

**Live example:** 900 sqft urban tiles basic → predicted ₹3,10,742 vs actual ₹3,13,700 = **0.94% error**

## Slide 4 — EDA: Distributions

- Cost: right-skewed, avg ₹10.9L, range ₹55K–₹50.5L
- Temp drop: bell curve, avg 8.1°C, range 5–12°C
- Units saved: right-skewed, avg 46 kWh/month ≈ ₹322/month at ₹7/kWh
- Dataset after cleaning: 2,472 rows

---

## Slide 5 — EDA: Categorical Analysis

- Roof types and cool roofing methods: balanced, 400–550 samples each
- Cost by location: dense urban ₹14L, urban ₹11L, intermediate ₹9L, rural ₹7L (2× gap)
- Cost by quality: basic ₹5L, intermediate ₹11L, high ₹18L (3.6× gap)
- **Quality = single strongest cost predictor**

---

## Slide 6 — ML Pipeline

- **Stage 1:** 2,500 rows, 15 columns raw data
- **Stage 2:** 106 missing values filled; 32 rows with temp=120°C (sensor error) → NaN; 9 features → 22 after one-hot encoding
- **Stage 3:** Split 70/15/15 (1,730 train / 371 val / 371 test), random_state=42
- **Stage 4:** RandomizedSearchCV 40 iterations × 5-fold CV = 200 fits per model
- **Stage 5:** Streamlit app, 5 models saved as .joblib, load in 50ms

---

## Slide 7 — Regression Results

- All 3 regression models vs baseline (baseline = always predict mean, $R^2 \approx 0$)
- Cost: $R^2$ 0.9843 — **98.4% variance explained**
- Temp drop: $R^2$ 0.935
- Units saved: $R^2$ 0.944
- Top features make physical sense (quality, building_size, avg_roof_temp_before) → no data leakage

---

## Slide 8 — Cost Model Deep Dive

- XGBoost: 2,000 trees, each corrects previous errors
- Training $R^2 \approx 0.997$, Test $R^2 = 0.9843$ (small gap = healthy, not overfit)
- MAE ₹69,426 vs Baseline MAE ₹6,08,878 → **8.8× better**
- RMSE ₹1,11,562 (higher than MAE = some large outlier errors, expected for heavy-tailed cost)
- Key hyperparams: lr=0.06 (stable learning), λ=0.5 (prevents overfit), subsample=0.9

---

## Slide 9 — Classification Results

- **High Rating:** 65.5% acc, AUC 0.728, F1 0.63 vs baseline 56.1%
- **Referral:** 62.0% acc, AUC 0.646, F1 0.505 vs baseline 57.1%
- Both use `class_weight='balanced'` — without it model just predicts majority class
- Referral lower because social behaviour (network, word-of-mouth) not in dataset

---

## Slide 10 — Customer Satisfaction

- Ratings: 1★ ~310, 2★ ~490, 3★ ~715 (mode), 4★ ~580, 5★ ~380
- 65% buildings got ≥4★ (positive outcome majority)
- Referral: ~36% refer (64% no referral) — 64:36 split more imbalanced than rating
- Mild imbalance handled with class_weight, not oversampling

---

## Slide 11 — Live Application

- **Home:** Live KPIs at startup, all 5 model status indicators
- **Dashboard:** 8 Plotly charts, 4 filters (location, quality, method, size) — all update simultaneously
- **Predict:** 9 inputs → 5 predictions + payback period = cost / (units_saved × tariff × 12)
- **Batch:** Upload CSV → 2,500 rows predicted in **0.23 seconds** (vectorised, no Python loops)
- **Recommender:** Ranks all 15 method × quality combos by weighted score (cost −0.25, temp +0.25, units +0.25, rating +0.15, referral +0.10)

---

## Slide 12 — Roadmap & Impact

- Q1 2025: FastAPI REST layer (mobile + third-party access)

- Q2 2025: PostgreSQL logs for audit trail + drift detection
- Q3 2025: Nightly auto-retraining on new installation records
- Q4 2025: IoT sensor integration for automatic label generation
- Impact: ₹70K accuracy pre-visit, 2.6-year payback at ₹7/kWh, 2,500 rows/0.3s batch

---

## Key Terms (Quick)

**$R^2$** — How much of the variation the model explains. 0 = useless, 1 = perfect. Our cost $R^2$ = 0.9843 = 98.4% explained.

**XGBoost** — Builds 2,000 trees sequentially; each tree fixes errors of the previous ones. Chosen because cost is right-skewed and non-linear.

**Random Forest** — Builds 300 trees independently; averages their predictions. Used for all other tasks.

**70/15/15 Split** — 1,730 train / 371 validate / 371 test rows. Fixed seed (random_state=42) = reproducible results.

**MAE** — Average absolute error. Our cost MAE = ₹69,426 vs baseline ₹6,08,878 (8.8× better).

**RMSE** — Like MAE but punishes large errors more. Cost RMSE = ₹1,11,562.

**ROC-AUC** — Classifier ranking quality. 0.5 = random, 1.0 = perfect. Rating: 0.728. Referral: 0.646.

**F1 Score** — Balance of precision and recall. Rating: 0.63. Referral: 0.505.

**class_weight='balanced'** — Prevents classifier from predicting majority class every time. Essential for 57:43 split.

**One-hot encoding** — Converts text categories to 0/1 columns. 9 inputs → 22 model features.

**Feature masking** — Zero out unwanted features instead of retraining. Enables Recommender to test 15 combos with 1 model.

**Regularisation λ=0.5** — Penalises large leaf weights in XGBoost. Prevents memorising training data.

**Vectorised inference** — NumPy array operations instead of Python loops. 18× faster. 2,500 rows in 0.23s.

**Baseline model** — Always predicts mean. MAE = ₹6.08L. We beat it 8.8×. If you don't beat baseline, model learned nothing.

---

*All values from test set (371 held-out rows, never seen during training)*